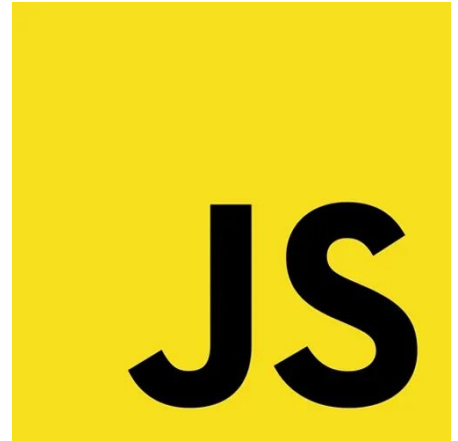


JavaScript Execution

The different ways JavaScript is run
TypeScript compilation
JavaScript versions



bitcore-build

bitcore-cli

bitcore-client

bitcore-cli, bitcore-client, bitcore-logging,**bitcore-wallet-client, bitcore-node, bitcore-wallet-service,
crypto-wallet-core, and insight use typescript**

bitcore-mnemonic

bitcore-node

bitcore-p2p

bitcore-p2p-cash

bitcore-p2p-doge

bitcore-tss

bitcore-wallet-client

bitcore-wallet-service

crypto-rpc

crypto-wallet-core

insight

≡ build

≡ pub

How does bitcore configure JavaScript?

TS

**bitcore-lib, bitcore-lib-cash, bitcore-lib-doge,
bitcore-lib-ltc, bitcore-mnemonic, bitcore-p2p,
bitcore-p2p-cash, bitcore-p2p-doge, bitcore-tss,
and crypto-rpc use vanilla JavaScript**

JS

Before going into more depth on configuration, some background is needed on CommonJS v.s. ECMAScript

CommonJS

ServerJS was a project created in 2009 to standardize module usage outside of web browsers. 7 months later it was renamed to CommonJS.

NodeJS originally used CommonJS ** (caveats???)

Browsers do not support CommonJS. However, CommonJS is frequently used in browsers via a transpiler

CommonJS - Technical

CommonJS modules are synchronous, meaning they execute and load files in a blocking manner. This approach works well in server environments where files are loaded from the local filesystem but can become problematic in browsers, where asynchronous execution is preferred.

Despite its popularity in Node.js, CommonJS has no static analysis, meaning tools cannot optimize code by removing unused functions (no tree shaking). It also relies on synchronous execution, which is inefficient for browser-based applications.

CommonJS uses `module.exports` and `require`

```
const bitcore = module.exports;
```

```
bitcore.crypto.ECDSA = require('./lib/crypto/ecdsa');
```

What is NodeJS?

CommonJS is JavaScript for the server, but isn't that what NodeJS is?

CommonJS is an import standard, NodeJS is a runtime environment.

NodeJS now supports ECMAScript

ECMAScript

ECMAScript is the original version of JavaScript announced in 1995

ECMAScript is used in browsers and servers

ECMAScript has a long history with many dramatic updates

The following slides will answer the following questions:

1 What was its first version like?

2 What are its advantages and limitations?

3 How is it different than CommonJS



ES

ECMAScript - History

The first version of ECMAScript was originally created for the Netscape browser in 1995. It had no module system, all JavaScript files were linked using <script> tags in HTML files. It lacked a lot of basic modern JavaScript.

ES3 (1999): try...catch, switch, do...while, and regex

ES5 (2009): Array, String, JSON, Date, Object, .bind, and “use strict”

ES6 (2015): let, const, Map, Set, () ->, { a, b } = operator, ... operator, for/of, classes, Promises, isFinite(), isNaN(), and modules

ECMAScript 2016: ** operator ECMAScript 2017: async/await

ECMAScript 2020: Dynamic import, BigInt, and import.meta

ECMAScript 2022: private methods and fields



ECMAScript - modules

Modules were introduced into ECMAScript in ES6, before multiple `<script>` in HTML were needed to have multiple JavaScript files

ECMAScript versions after ES6 use

- Uses `import` and `export` instead of `require()` and `module.exports`.
- Enables static analysis, allowing tree shaking (removal of unused code).
- Supports top-level `await` - Requires explicit file extensions in relative imports (`.mjs`, `.js`, or `.cjs`).

To use ESM in Node.js, update `package.json` by adding `"type": "module"`. This ensures that `.js` files are treated as ES modules. If you prefer not to modify the project-wide setting, rename files to `.mjs` instead.

ECMAScript - TypeScript Configuration

tsc needs to know what kind module system the TypeScript is used and what kind of JavaScript to create

target: output JavaScript version (es6, es2022, etc.)

module: output module system

moduleResolution: input module system

lib: somehow different from target